

```
-- 1. Calculate account metrics.  
-- Since the account table does not contain account creation date or country,  
-- the first recorded session is used as a proxy.
```

```
WITH account_metrics AS (  
  SELECT  
    date,  
    country,  
    send_interval,  
    is_verified,  
    is_unsubscribed,  
    COUNT(DISTINCT account_id) AS account_cnt,  
    0 AS sent_msg,  
    0 AS open_msg,  
    0 AS visit_msg  
  FROM (  
    SELECT  
      acs.account_id,  
      DATE(s.date) AS date,  
      sp.country AS country,  
      a.send_interval AS send_interval,  
      a.is_verified AS is_verified,  
      a.is_unsubscribed AS is_unsubscribed,  
      ROW_NUMBER() OVER (  
        PARTITION BY acs.account_id  
        ORDER BY s.date, acs.ga_session_id  
      ) AS rn  
    FROM `DA.account` a  
    JOIN `DA.account_session` acs  
      ON a.id = acs.account_id  
    JOIN `DA.session` s  
      ON acs.ga_session_id = s.ga_session_id  
    JOIN `DA.session_params` sp  
      ON acs.ga_session_id = sp.ga_session_id  
    WHERE s.date IS NOT NULL  
  )  
)
```

```

WHERE rn = 1
GROUP BY
    date,
    country,
    send_interval,
    is_verified,
    is_unsubscribed
),

```

```

-- 2. Prepare one session and one country per account for email metrics.
-- The earliest recorded session is used as a proxy to avoid duplicating
-- the same email across multiple sessions of the same account.

```

```

email_base AS (
SELECT
    account_id,
    s.date AS session_date,
    sp.country AS country
FROM (
    SELECT
        acs.account_id,
        acs.ga_session_id,
        ROW_NUMBER() OVER (
            PARTITION BY acs.account_id
            ORDER BY s.date, acs.ga_session_id
        ) AS rn
    FROM `DA.account_session` acs
    JOIN `DA.session` s
        ON acs.ga_session_id = s.ga_session_id
    WHERE s.date IS NOT NULL
) x
JOIN `DA.session` s
    ON x.ga_session_id = s.ga_session_id
JOIN `DA.session_params` sp
    ON x.ga_session_id = sp.ga_session_id
WHERE x.rn = 1

```

```
),
```

```
-- 3. Calculate email metrics.
```

```
-- sent_date is treated as a day index and converted into a proper date
```

```
-- by adding it to the session date.
```

```
email_metrics AS (  
  SELECT  
    DATE_ADD(eb.session_date, INTERVAL ems.sent_date DAY) AS date,  
    eb.country AS country,  
    a.send_interval AS send_interval,  
    a.is_verified AS is_verified,  
    a.is_unsubscribed AS is_unsubscribed,  
    0 AS account_cnt,  
    COUNT(DISTINCT ems.id_message) AS sent_msg,  
    COUNT(DISTINCT eo.id_message) AS open_msg,  
    COUNT(DISTINCT ev.id_message) AS visit_msg  
  FROM `DA.email_sent` ems  
  JOIN `DA.account` a  
    ON ems.id_account = a.id  
  JOIN email_base eb  
    ON ems.id_account = eb.account_id  
  LEFT JOIN `DA.email_open` eo  
    ON ems.id_message = eo.id_message  
  LEFT JOIN `DA.email_visit` ev  
    ON ems.id_message = ev.id_message  
  GROUP BY  
    DATE_ADD(eb.session_date, INTERVAL ems.sent_date DAY),  
    eb.country,  
    a.send_interval,  
    a.is_verified,  
    a.is_unsubscribed  
) ,
```

```
-- 4. Combine account and email metrics into one dataset.
```

```
unioned_data AS (  
  SELECT  
    date,  
    country,  
    send_interval,  
    is_verified,  
    is_unsubscribed,  
    account_cnt,  
    sent_msg,  
    open_msg,  
    visit_msg  
  FROM account_metrics  
  
  UNION ALL  
  
  SELECT  
    date,  
    country,  
    send_interval,  
    is_verified,  
    is_unsubscribed,  
    account_cnt,  
    sent_msg,  
    open_msg,  
    visit_msg  
  FROM email_metrics  
)
```

-- 5. Aggregate all metrics by the required dimensions.

```
final_agg AS (  
  SELECT  
    date,  
    country,  
    send_interval,  
    is_verified,
```

```

        is_unsubscribed,
        SUM(account_cnt) AS account_cnt,
        SUM(sent_msg) AS sent_msg,
        SUM(open_msg) AS open_msg,
        SUM(visit_msg) AS visit_msg
FROM unioned_data
GROUP BY
    date,
    country,
    send_interval,
    is_verified,
    is_unsubscribed
),

```

-- 6. Calculate country totals.

```

country_summary AS (
SELECT
    country,
    SUM(account_cnt) AS total_country_account_cnt,
    SUM(sent_msg) AS total_country_sent_cnt
FROM final_agg
GROUP BY country
),

```

-- 7. Rank countries by total accounts and total sent messages.

```

country_ranks AS (
SELECT
    country,
    total_country_account_cnt,
    total_country_sent_cnt,
    RANK() OVER (ORDER BY total_country_account_cnt DESC) AS
rank_total_country_account_cnt,
    RANK() OVER (ORDER BY total_country_sent_cnt DESC) AS rank_total_country_sent_cnt
FROM country_summary

```

)

-- 8. Return the final dataset and keep only top 10 countries
-- by total accounts or by total sent messages.

```
SELECT
  f.date,
  f.country,
  f.send_interval,
  f.is_verified,
  f.is_unsubscribed,
  f.account_cnt,
  f.sent_msg,
  f.open_msg,
  f.visit_msg,
  cr.total_country_account_cnt,
  cr.total_country_sent_cnt,
  cr.rank_total_country_account_cnt,
  cr.rank_total_country_sent_cnt
FROM final_agg f
JOIN country_ranks cr
  ON f.country = cr.country
WHERE cr.rank_total_country_account_cnt <= 10
      OR cr.rank_total_country_sent_cnt <= 10
ORDER BY f.date, f.country;
```

ADVANCED MODULE TASK

	country	account_cnt ▾	total_country_sent_cnt	rank_total_country_acc...	rank_total_count...
1.	United States	12,384	179,330,304	1	1
2.	India	2,687	28,768,218	2	2
3.	Canada	2,067	19,829,022	3	3
4.	United Kingdom	859	6,078,728	4	4
5.	France	553	3,570,000	5	6
6.	Spain	536	2,989,665	6	7
7.	Taiwan	500	2,793,456	7	9
8.	Germany	490	3,119,907	8	8
9.	China	490	3,227,377	8	5
10.	Italy	386	2,771,847	10	11

1 - 11 / 11 < >

